

Layer_6

Layer 6: The Presentation Layer — ASN.1 (Examinable Scope)

Scope note: For Semester Test 2, **only ASN.1** is examinable from Layer 6. The other Layer 6 topics (character sets, UTF-8/16/32 encoding, MIME, compression, encryption, etc.) are **not in scope** for this test. They are retained briefly only where they directly affect ASN.1's context. The original character-encoding and MIME notes from the broader Layer 6 chapter have been removed from this scope-trimmed file.

1. What ASN.1 Is

Abstract Syntax Notation One (ASN.1) is a notation used by some protocols to specify the **syntax of messages** at the presentation layer. In many ways it works like a record or structure declaration in a programming language: a message consists of a number of named fields, and some of those fields may themselves be records (i.e. structured).

ASN.1 was originally introduced as the "complexity extreme" of message specification, in contrast to lightweight, text-based, informally specified protocols like POP3 (which uses simple keywords such as `RETR msg` followed by an English description of the argument's meaning). ASN.1 instead uses a **strict formal grammar** to define message structure.

Key examination facts about ASN.1

- **The syntactic structure of an ASN.1 message is defined using a grammar** (not a natural-language description, not a diagram, not a bit pattern, and not a list of component types).
 - ASN.1 separates **abstract syntax** (what the data structure looks like) from **physical encoding** (how the bits are actually laid out on the wire). The same ASN.1 description can be encoded in several different ways depending on which encoding rules are chosen — see Section 3 below.
 - Because the sender and receiver both know the abstract syntax and the encoding used, **tags are not necessarily required to delimit fields**. This is one of the main reasons ASN.1 messages can be far more compact than XML.
-

2. ASN.1 Data Types and Constructors

ASN.1 provides a number of **standard built-in types** such as character strings, integers, real numbers, and some more esoteric types.

Built-in (primitive) types include:

- `INTEGER` — a whole number, encoded as binary or as ASCII digits depending on the encoding rules.
- `VisibleString` — a printable character string.
- `OCTET STRING` — a sequence of arbitrary 8-bit bytes.
- (Other primitive types exist; for the exam you only need to recognise the names.)

Constructors (structuring keywords):

ASN.1 builds compound types out of primitives using **constructors**:

- `SEQUENCE` — an **ordered** list of fields (like a `struct` in C). Used, for example, for a student's biographical details where the order of `surname`, `initials`, `address` matters.
- `SET` — an **unordered** collection of values. Used, for example, for the set of all courses a student has completed (the order in which courses appear is immaterial).
- `CHOICE` — selects one of several alternatives. Used, for example, where an address may be either a *street address* or a *PO box*, and the two cases carry different sub-fields.

What is **NOT** a valid ASN.1 type / constructor

The following names are **invalid** in ASN.1 (commonly used as MCQ distractors):

- **✗** `STRUCT` — looks like a programming-language keyword, but ASN.1 uses `SEQUENCE` instead.
- **✗** `ARRAY` — also not an ASN.1 keyword; use `SET OF` or `SEQUENCE OF` for collections.
- **✗** `MESSAGE` — never an ASN.1 type.

MCQ pattern: "Which of the following is *not* a valid data type / constructor in ASN.1?" — the answer is whichever of `STRUCT`, `ARRAY`, or `MESSAGE` appears. The valid options (`SEQUENCE`, `SET`, `CHOICE`, `INTEGER`) should never be picked.

Example: an academic record in ASN.1

The textbook gives the canonical example for a list of courses:

```
Courses ::= SET
{ code      VisibleString,
```

```
courseName  VisibleString,
mark        INTEGER
-- Other fields to be added here
}
```

An actual *value* matching this declaration:

```
{ {code "COS332", courseName "Networks", mark 55},
  {code "COS301", courseName "Project", mark 91}
}
```

A student record would typically be a `SEQUENCE` of biographical details, one of whose fields is a `SET` of `Courses`, and where an address sub-field might be a `CHOICE` between a street address and a PO Box.

Capitalisation and whitespace

- Capitalisation **is important** in ASN.1, but the COS332 module is lenient about it for marking purposes.
- White space is generally not significant.
- The formal ASN.1 standard is **146 pages long** — part of why ASN.1 is sometimes seen as complex and avoided by practitioners.

3. ASN.1 Encoding Rules

Once a message has been described abstractly in ASN.1, it still has to be **encoded** as a sequence of bytes on the wire. ASN.1 deliberately separates the abstract specification from the encoding, and several encoding-rule standards exist:

Encoding Rules	Acronym	Brief description
Basic Encoding Rules	BER	The original, most flexible encoding. Each value is encoded as Type, Length, Value (TLV).
Canonical Encoding Rules	CER	Like BER, but removes some of the alternative options BER allows. Produces a single canonical encoding.
Distinguished Encoding Rules	DER	Like CER, but where possible uses fixed lengths for types, so the length field can sometimes be omitted. Trades flexibility for efficiency.
Packed Encoding Rules	PER	Even more compact; omits redundant information.

Encoding Rules	Acronym	Brief description
Encoding Control Notation	ECN	Lets a designer customise how encoding is done.
XML Encoding Rules	XER	Encodes ASN.1 data as XML. Recovers human readability at the cost of larger message sizes.

BER in detail (the most exam-relevant)

MCQ key fact: A BER value in ASN.1 is, in principle, encoded as a **triple consisting of a type, a length, and a value** (i.e. TLV).

- The **Type** is an octet at the start of any value, with various bits set or reset to describe the data that follows.
- The **Length** follows the type octet and specifies how many bytes of value data follow.
- The **Value** is the actual data in its native format — an `INTEGER` is sent as binary, characters in the agreed character code, and so on.

In a sense the data is therefore "marked up" after all — but the markup is binary and not human-readable.

How DER and CER relate to BER

- **CER** = BER minus the alternative-option flexibility — one canonical form per value.
- **DER** = CER plus fixed lengths where possible (so the length field may be implicit). This is *less* flexible but progressively *more* efficient.

4. ASN.1 vs XML (Why Use ASN.1?)

ASN.1 and XML schemas are similar in that both are used to describe the syntax of well-formed messages. The most important difference:

	XML	ASN.1
Specifies tags?	Yes — fields are wrapped in named tags	No — tags not required; sender and receiver share abstract syntax
Field data	Textual representation	Native binary (or whatever encoding rule prescribes)
Human-readable?	Yes — easy to inspect a message and locate problems	No — typical ASN.1 message contains lots of binary and no obvious delimiters

	XML	ASN.1
Bandwidth cost	High — bloated by tags and textual values	Low — much more compact, less resource-intensive
Markup language?	Yes	No (it is purely an abstract specification)

So XML is human-readable but consumes excessive bandwidth; ASN.1 is bandwidth-efficient and processing-efficient but hard for a human to inspect.

Important context: XML's `XML Schemas` and ASN.1 play very similar roles (both describe well-formed message syntax). The key differentiator is *abstract vs concrete*: ASN.1 abstracts what the data looks like and lets encoding rules handle the physical layout; XML directly mandates the on-the-wire form (with tags and text).

5. Real-World Uses of ASN.1

ASN.1 is widely deployed even though most users never see it:

- **X.509 certificates** — defined in ASN.1. The examples of certificates seen in Layer 3 / TLS material are ASN.1 values.
- **SNMP (Simple Network Management Protocol)** — specifies messages in ASN.1 and **requires BER encoding** to be used.
- **PKCS #8** (RFC 5208) — describes the syntax of a private key using ASN.1. Prescribes **BER** encoding.
- **PKCS #10** (RFC 2986) — describes the syntax of a certification request using ASN.1. Prescribes **DER** encoding.
- **LDAP** (RFC 4511) — *all* LDAP messages are "transferred using a subset of ASN.1 Basic Encoding Rules ([BER])." Practically this means LDAP messages: use an octet as type descriptor at the start of any value; encode the actual data in its native format (e.g. binary for `INTEGER`); and the length of the actual data is *not* always explicitly specified in every variant (since LDAP uses a subset).

Example PKCS #8 (from textbook Figure 4.2)

```
PrivateKeyInfo ::= SEQUENCE {
    version             Version,
    privateKeyAlgorithm AlgorithmIdentifier {{PrivateKeyAlgorithms}},
    privateKey          PrivateKey,
    attributes          [0] Attributes OPTIONAL }
```

```
Version ::= INTEGER {v1(0)} (v1,...)
PrivateKey ::= OCTET STRING
Attributes ::= SET OF Attribute
```

Note the use of:

- `SEQUENCE` for the ordered fields of the private-key info.
- `OPTIONAL` to mark the `attributes` field as optional.
- `INTEGER` constrained to specific named values.
- `SET OF` to denote an unordered collection of `Attribute` values.

6. Why ASN.1 sits on Layer 6

The presentation layer's job is to encode/decode information so that data is correctly represented in transit and accurately reconstructed at the destination. ASN.1 fits squarely on Layer 6 because it:

1. **Separates how the application thinks about a message** (the abstract syntax, the academic-record `SEQUENCE`, etc.) **from how the message is laid out on the wire** (BER/DER/etc.). That separation is the essence of presentation-layer functionality.
2. Provides **endpoint-to-endpoint** encoding agreement — both sides must share the abstract syntax and the encoding rule, exactly like other presentation-layer mechanisms.

This is also why protocols like SNMP, LDAP, and the X.509/PKCS family — all of which need precise, language-independent, compact message representations — chose ASN.1.

Examinable Practice Questions on ASN.1

Multiple Choice

Q1. A BER value in ASN.1 is, in principle, encoded as a triple consisting of:

- A. A type, a subtype, and a value
- B. A type, a length, and a value
- C. A constructor and two operands
- D. A variable name, as well as its minimum and maximum values
- E. The same value encoded in binary, text, and hexadecimal

Q2. Which of the following is *not* a valid data type / constructor in ASN.1?

- A. STRUCT
- B. CHOICE
- C. SEQUENCE
- D. ARRAY
- E. INTEGER

Q3. The syntactic structure of an ASN.1 message is defined using:

- A. A grammar
- B. An informal description in natural language
- C. A diagrammatic depiction of the message
- D. A bit pattern
- E. A list of the types of the components of the message

Q4. (2024-style) Which of the following is a valid ASN.1 *encoding*?

- A. BER
- B. ASCII
- C. EBCDIC
- D. ISO OSI
- E. Grammar

Q5. (2024-style) Which of the following is *not* a valid ASN.1 constructor?

- A. SEQUENCE
- B. MESSAGE
- C. CHOICE
- D. SET

Q6. (Final-exam style, 2024) "According to RFC 4511, all LDAP messages are transferred using a subset of ASN.1 Basic Encoding Rules ([BER])." This means LDAP messages will:

- A. Use an octet as a type descriptor at the start of any value
- B. The length of the actual data will be explicitly specified
- C. The actual data will be encoded in its native format (such as binary for integers)
- D. More than one of the above
- E. All of the above

Long-Form

Q7. Two distinct software systems need to securely and efficiently exchange complex academic records consisting of biographical details and nested course information. System A proposes formatting the data using XML, while System B proposes using ASN.1 with Distinguished Encoding Rules (DER). From a **bandwidth** and **processing-efficiency** perspective, which method is more efficient and why?

Q8. Briefly state the difference between **BER**, **CER**, and **DER** as ASN.1 encoding rules.

Q9. Provide a small ASN.1 declaration that defines a `Person` record consisting of:

- A surname (string)
- An age (integer)
- A list of email addresses (any number, order does not matter)
- An address that is *either* a street address (`StreetAddr`) or a PO box (`POBox`)

You may assume `StreetAddr` and `POBox` are defined elsewhere.

Memo / Answer Key

A1. B — A type, a length, and a value (TLV).

A2. Both **A (STRUCT)** and **D (ARRAY)** are invalid. The 2025 ST2 paper accepted ARRAY as the intended answer (the question explicitly used the wording "STRUCT" or "ARRAY" depending on variant; both are correct rejections). Make sure to pick whichever of STRUCT / ARRAY / MESSAGE appears in *your* paper.

A3. A — A grammar.

A4. A — BER. ASCII, EBCDIC, and ISO OSI are not ASN.1 encoding rules. A grammar is what *defines* ASN.1's syntax, not how a value is encoded.

A5. B — MESSAGE. (SEQUENCE, CHOICE, and SET are all valid constructors.)

A6. D — More than one of the above. LDAP messages do use an octet as a type descriptor at the start of any value (A is correct), and the actual data is encoded in its native format such as binary for integers (C is correct). However, because LDAP uses a *subset* of BER, the explicit length specification (B) is not always present — so B is not strictly correct. The textbook/lecturer-intended answer here is the combination of A + C, so the option that captures "more than one" is correct.

A7. System B (ASN.1 + DER) is vastly more efficient. XML relies on textual, human-readable tags and textual representations of values, which consume large amounts of bandwidth and require text parsing. ASN.1 separates the abstract syntax from the encoding; DER (a stricter variant of BER) then encodes values directly in compact binary, omits length fields where the length is fixed and known, and uses no textual delimiters at all. The result is smaller messages on the wire and far cheaper parsing at the endpoint. The trade-off is that an ASN.1/DER message is essentially impossible for a human to inspect by eye, whereas an XML message can be opened in any text editor.

A8.

- **BER (Basic Encoding Rules):** the original, most flexible. Every value is encoded as Type-Length-Value (TLV). BER permits multiple valid encodings of the same abstract value.
- **CER (Canonical Encoding Rules):** like BER, but removes the alternative options, so a given abstract value has *one* canonical encoding.
- **DER (Distinguished Encoding Rules):** like CER, but where possible uses *fixed* lengths for types, so the explicit length field can be omitted. DER is stricter than CER and produces the smallest of the three encodings — at the cost of flexibility.

A9. A reasonable ASN.1 declaration:

```
Person ::= SEQUENCE {  
    surname    VisibleString,  
    age        INTEGER,  
    emails     SET OF VisibleString,  
    address    CHOICE { street  StreetAddr,  
                        poBox    POBox }  
}
```

Key points being marked: `SEQUENCE` for the ordered top-level fields, `SET OF` for the unordered list of emails, `CHOICE` for the either-or address. Order of fields inside `SEQUENCE` is not strictly material for marking, but order matters in ASN.1 semantics.